

Conditional Workflows in LangGraph

Branching, routing functions, and building graphs where execution takes exactly one of several possible paths.

WHAT THIS CHAPTER COVERS

- ▶ What a conditional workflow is
- ▶ Conditional vs. sequential vs. parallel
- ▶ Routing functions and Literal return types
- ▶ `graph.add_conditional_edges()` in depth
- ▶ Case study: quadratic equation solver
- ▶ Case study: LLM review-handling workflow
- ▶ Structured output for routing decisions
- ▶ Common mistakes in branch-based logic

Conditional Workflows in LangGraph

RECAP — CHAPTERS 5 & 6

Chapter 5 covered **sequential workflows** — a fixed series of nodes executed strictly one after another. Chapter 6 covered **parallel workflows** — multiple branches launched together from a fan-out point and merged back at a fan-in node, all branches executing on every run. This chapter introduces the third fundamental workflow pattern: **conditional workflows**, where the graph must choose exactly one branch to execute out of several possible options.

1. Introducing Conditional Workflows

Definition

A *conditional workflow* is a graph in which execution reaches a decision point after which the graph follows **exactly one** of several possible branches, chosen at runtime based on a condition evaluated against the current state. This is the LangGraph equivalent of an `if / elif / else` statement in ordinary programming, except the "statements" are entire nodes (or sub-graphs) rather than single lines of code.

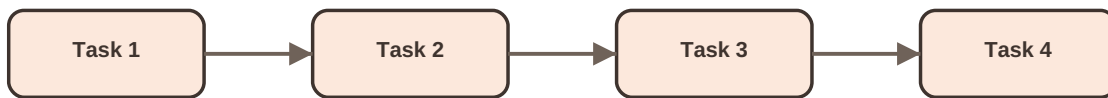
Conditional vs. Parallel — The Critical Distinction

A conditional workflow can look structurally similar to a parallel workflow on paper — both have a single node branching out to multiple downstream nodes. The difference is entirely in **how many of those branches actually run**:

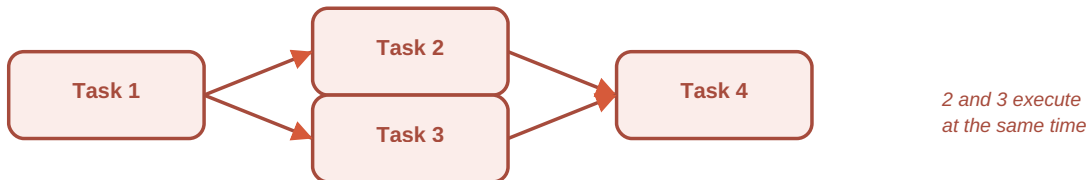
- In a **parallel workflow**, every branch coming out of the fan-out node executes on every run.
- In a **conditional workflow**, only **one** branch executes per run — the others are skipped entirely.

Figure 7.1 places all three patterns side by side so the distinction is visually unmistakable.

SEQUENTIAL — one path, one order



PARALLEL — multiple branches run together



CONDITIONAL — exactly one branch is chosen

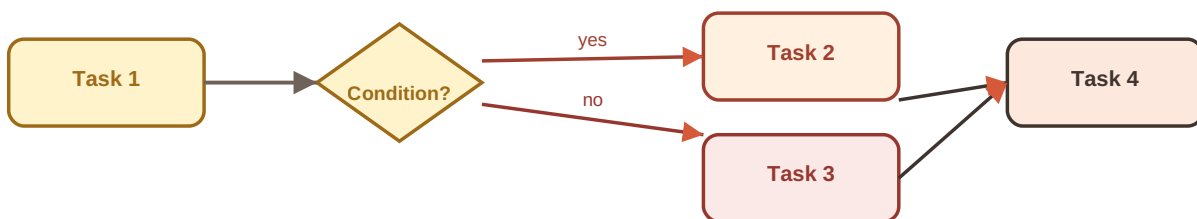


Figure 7.1 — Sequential execution runs one path in order; parallel execution runs multiple branches together; conditional execution routes to exactly one branch.

Why It Exists / Problem It Solves

Real agentic systems constantly need to make decisions: is a customer review positive or negative? Does a value satisfy a threshold? Should a tool be called, or is the answer already known? Without conditional routing, every graph would be forced into a single rigid execution order, which cannot model any real decision-making logic. Conditional workflows give a graph the ability to adapt its own execution path to the data it is currently holding in state.

WHEN YOU WILL NEED THIS

- Sentiment-based or classification-based routing (e.g. positive vs. negative review handling).
- Mathematical or business-rule branching (e.g. discriminant sign, threshold checks, tier-based logic).
- Almost every non-trivial agentic graph built later in this playlist will use conditional edges — it is one of the most frequently used building blocks in LangGraph, on par with `if / else` in general-purpose programming.

IMPORTANT

A conditional workflow is not limited to exactly two branches. LangGraph places no hard limit on the number of branches a routing function can return — there can be two, three, or many more, exactly as with a chained `if / elif / elif / else`.

🔄 Quick Revision — Conditional Workflows

- A conditional workflow routes execution to exactly one branch at runtime, based on the current state.
- It differs from a parallel workflow, where every branch executes on every run.
- Conditional routing is the LangGraph equivalent of `if / elif / else`, but at the level of whole nodes.
- There is no limit on the number of branches a conditional workflow can have.
- This pattern recurs constantly in agentic graphs — e.g. sentiment routing, threshold checks, rule-based branching.

2. The Routing Function and `add_conditional_edges()`

Definition

A *routing function* is a plain Python function that receives the current state and returns a **string** identifying which node should execute next. It is not registered as a node itself — it is registered separately as the decision logic for a conditional edge, using `graph.add_conditional_edges()`.

How It Works

The routing function's return value must exactly match the name under which a candidate destination node was registered with `graph.add_node(...)`. LangGraph looks at the string the routing function returns and sends execution straight to the node registered under that name — no other candidate branch is touched.

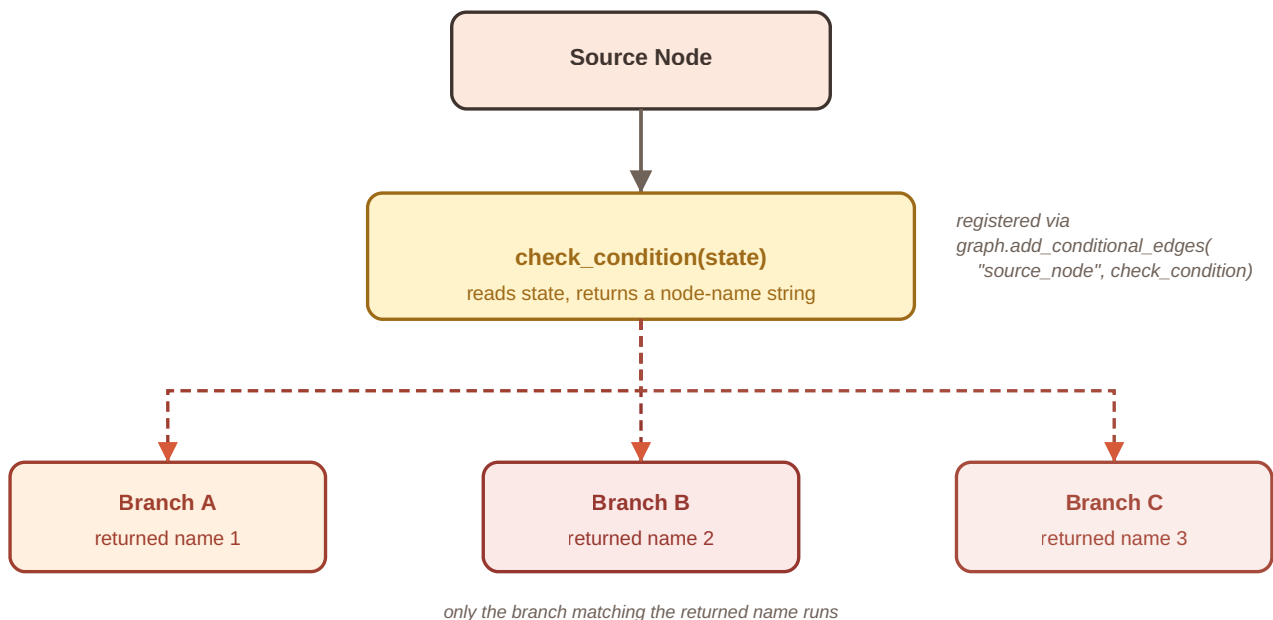


Figure 7.2 — A routing function inspects state and returns the name of exactly one downstream node; only the matching branch executes.

Anatomy of `add_conditional_edges()`

`graph.add_conditional_edges(source_node_name, routing_function)` takes two arguments: the name of the node the condition is attached to, and the routing function that decides where to go next. Internally, `LangGraph` calls the routing function with the current state immediately after `source_node_name` finishes executing, then follows whichever node name comes back.

TYPING THE ROUTING FUNCTION WITH LITERAL

It is strong practice to annotate the return type of a routing function using `typing.Literal`, listing every possible node name it can return. This does not change runtime behavior, but it gives static type checkers and IDEs a precise, self-documenting contract for every valid branch:

```
def check_condition(state: QuadState) -> Literal['real_roots', 'repeated_roots', 'no_roots']:
    if state['D'] > 0:
        return 'real_roots'
    elif state['D'] == 0:
        return 'repeated_roots'
    else:
        return 'no_roots'
```

BEST PRACTICE

Always give a routing function a `Literal[...]` return-type annotation listing every possible destination node name. It documents the branch contract directly in the function signature and catches typos in branch names during development rather than at runtime.

INTERVIEW TIP

If asked how conditional edges differ from ordinary edges in `LangGraph`, lead with: an ordinary edge is a fixed, unconditional connection created with `add_edge()`; a conditional edge is created with `add_conditional_edges()` and delegates the destination choice, at runtime, to a routing function that reads the current state and returns a node name.

🔄 Quick Revision — Routing Functions

- A routing function takes state as input and returns the string name of the next node to execute.
- It is registered separately from ordinary nodes, via `graph.add_conditional_edges(source, routing_fn)`.
- Its return value must exactly match a node name already registered with `add_node()`.
- Annotate its return type with `Literal[...]` listing every valid destination node name.
- There is a second, more advanced way to build conditional routing using the `Command` object, covered when dynamic workflows are introduced later in this playlist.

3. Case Study: The Quadratic Equation Workflow (Non-LLM)

The Problem: Solving a Quadratic Equation

A quadratic equation has the general form $ax^2 + bx + c = 0$. Its roots depend entirely on the *discriminant*, $D = b^2 - 4ac$:

Discriminant	Root Type	Formula
$D > 0$	Two distinct real roots	$(-b \pm \sqrt{D}) / 2a$
$D = 0$	One repeated real root	$-b / 2a$
$D < 0$	No real roots	—

Exactly one of these three cases applies to any given equation — which makes this problem a clean, self-contained example of conditional branching before any LLM is introduced.

Designing the State

The workflow's state, `QuadState`, needs to hold the three coefficients, the discriminant, the printable equation string, and the final result string:

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Literal

class QuadState(TypedDict):
    a : int
    b : int
    c : int
    D : int
    result : str
    equation : str
```

Node Functions

Two ordinary nodes run before the branching point: one to render the equation as a string, and one to compute the discriminant.

```
def show_equation(state : QuadState):
    a = state['a']
    b = state['b']
    c = state['c']

    equation = f"{a}x2 + ({b}x) + ({c})"

    return {"equation":equation}

def find_discriminant(state : QuadState):
    a = state['a']
    b = state['b']
    c = state['c']

    D = b**2 - 4*a*c

    return {'D':D}
```

Then come the three branch nodes, one per discriminant case, followed by the routing function that decides between them:

```
def find_real_roots(state : QuadState):
    a = state['a']
    b = state['b']
    D = state['D']

    root1 = (-b + (D**0.5))/2*a
    root2 = (-b - (D**0.5))/2*a

    return {'result':f"The roots are {root1} and {root2}"}

def find_repeated_roots(state : QuadState):
    root = -state['b']/(2*state['a'])

    return {'result':f"The only root is {root}"}

def return_no_roots(state : QuadState):

    return {'result':"No real roots"}

# Routing Function gives the names of upcoming nodes as output
def check_condition(state : QuadState) -> Literal['real_roots','repeated_roots','no_roots']:

    if state['D'] > 0:
        return 'real_roots'
    elif state['D'] == 0:
        return 'repeated_roots'
    else:
        return 'no_roots'
```

COMMON MISTAKE

Look closely at `find_real_roots`: `root1 = (-b + (D**0.5))/2*a`. Because Python evaluates `/` and `*` left-to-right with equal precedence, this expression is actually `((-b + √D) / 2) * a` — **not** `(-b + √D) / (2a)` as the quadratic formula requires. Whenever a formula involves dividing by a product like `2a`, wrap the denominator in explicit parentheses: `(-b + (D**0.5)) / (2*a)`. This is one of the most common silent bugs when translating a math formula directly into code.

🔄 Quick Revision — Quadratic Equation State & Nodes

- `QuadState` carries `a`, `b`, `c`, `D`, `equation`, and `result`.
- `show_equation` and `find_discriminant` are ordinary sequential nodes that run before the branch point.
- Three branch nodes exist: `real_roots`, `repeated_roots`, `no_roots` — only one executes per run.
- `check_condition` is the routing function, typed to return `Literal['real_roots','repeated_roots','no_roots']`.
- Watch operator precedence carefully when translating a math formula with a compound denominator into code.

4. Assembling, Compiling, and Running the Workflow

Assembling and Compiling the Graph

With all five nodes and the routing function defined, the graph is wired together. Two ordinary edges connect `START` through the linear part of the flow; one call to `add_conditional_edges()` replaces what would otherwise have needed three separate `add_edge()` calls with manual if-logic outside the graph; and three ordinary edges connect each branch back to `END`.

```
graph = StateGraph(QuadState)

graph.add_node("show_equation", show_equation)
graph.add_node("find_discriminant", find_discriminant)
graph.add_node("real_roots", find_real_roots)
graph.add_node("repeated_roots", find_repeated_roots)
graph.add_node("no_roots", return_no_roots)

graph.add_edge(START, "show_equation")
graph.add_edge("show_equation", "find_discriminant")

# Conditional Edge
graph.add_conditional_edges("find_discriminant", check_condition)

graph.add_edge("real_roots", END)
graph.add_edge("repeated_roots", END)
graph.add_edge("no_roots", END)

workflow = graph.compile()
```

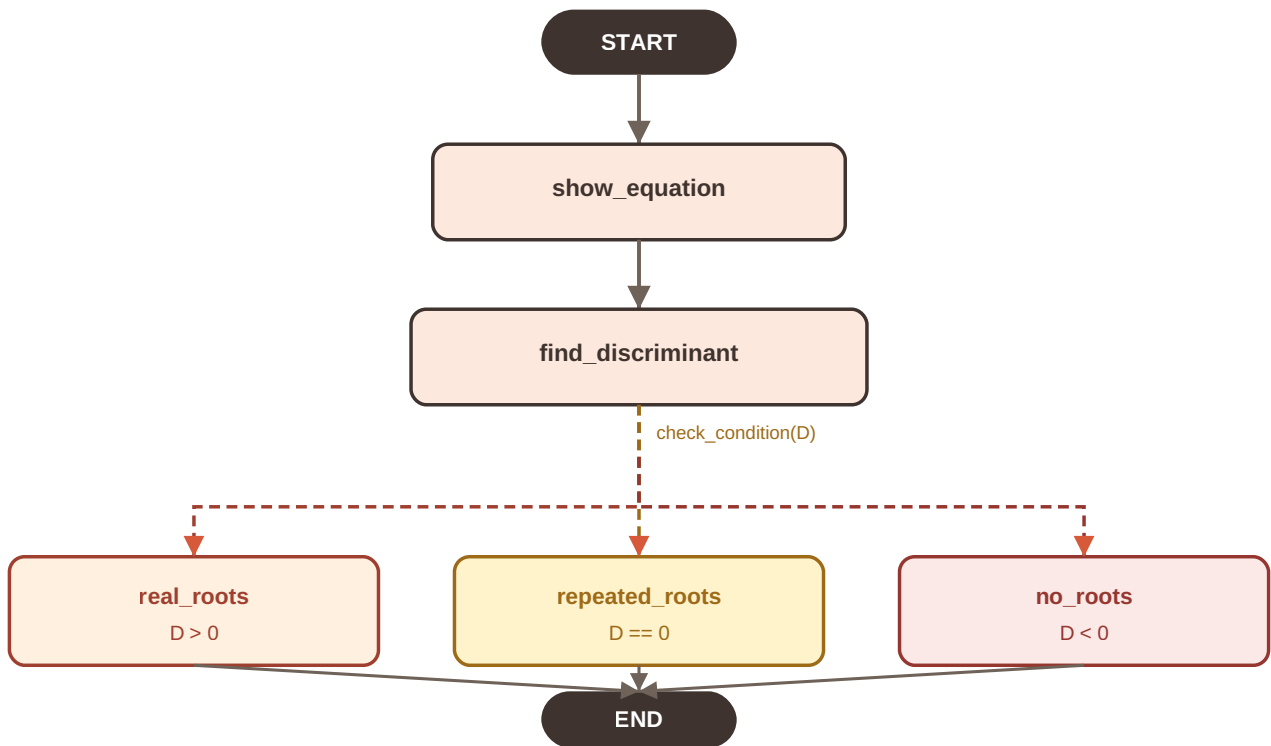



Figure 7.3 — Compiled topology of the quadratic-equation workflow: two sequential nodes feed into a single conditional edge with three mutually exclusive branches.

Running the Workflow

Invoking the compiled workflow with different coefficient values exercises all three branches:

```
initial_state = {
  'a' : 3,
  'b' : 4,
  'c' : 2
}

workflow.invoke(initial_state)
```

For **a=3, b=4, c=2**: $D = 4^2 - 4(3)(2) = 16 - 24 = -8$, which is negative, so the **no_roots** branch executes and the workflow reports no real roots. Changing the coefficients so that D becomes positive or exactly zero instead routes execution through **real_roots** or **repeated_roots** respectively — the rest of the graph never runs those other two branches on that particular invocation.

REAL WORLD INSIGHT

Notice that all three branch nodes converge back onto the same **END** target. This fan-in-after-branch shape is extremely common: conditional workflows frequently reconverge onto a shared downstream step (or straight to **END**) once the branch-specific work is done, so the rest of the graph doesn't need to know which branch was taken.

Quick Revision — Graph Assembly

- One call to `add_conditional_edges(source, routing_fn)` replaces three separate manual branch edges.
- All ordinary (unconditional) transitions still use `add_edge()` as usual.
- Every branch node in this workflow independently connects to END — a common convergence pattern.
- Only the branch matching the routing function's returned name executes on any given invocation.

5. Case Study: LLM-Based Review-Handling Workflow

The Problem: Responding to Customer Reviews

The second case study builds a more realistic, LLM-powered conditional workflow: automatically drafting a reply to a customer review. The routing decision this time is not a numeric comparison but a **sentiment classification** produced by an LLM call.

Planned Flow

- A review arrives as plain text.
- An LLM call extracts its sentiment (**positive** or **negative**) as structured output.
- If positive → a warm thank-you reply is generated directly.
- If negative → a deeper diagnosis step first extracts issue type, tone, and urgency (again as structured output), and only then is an empathetic, context-aware reply generated using those details.

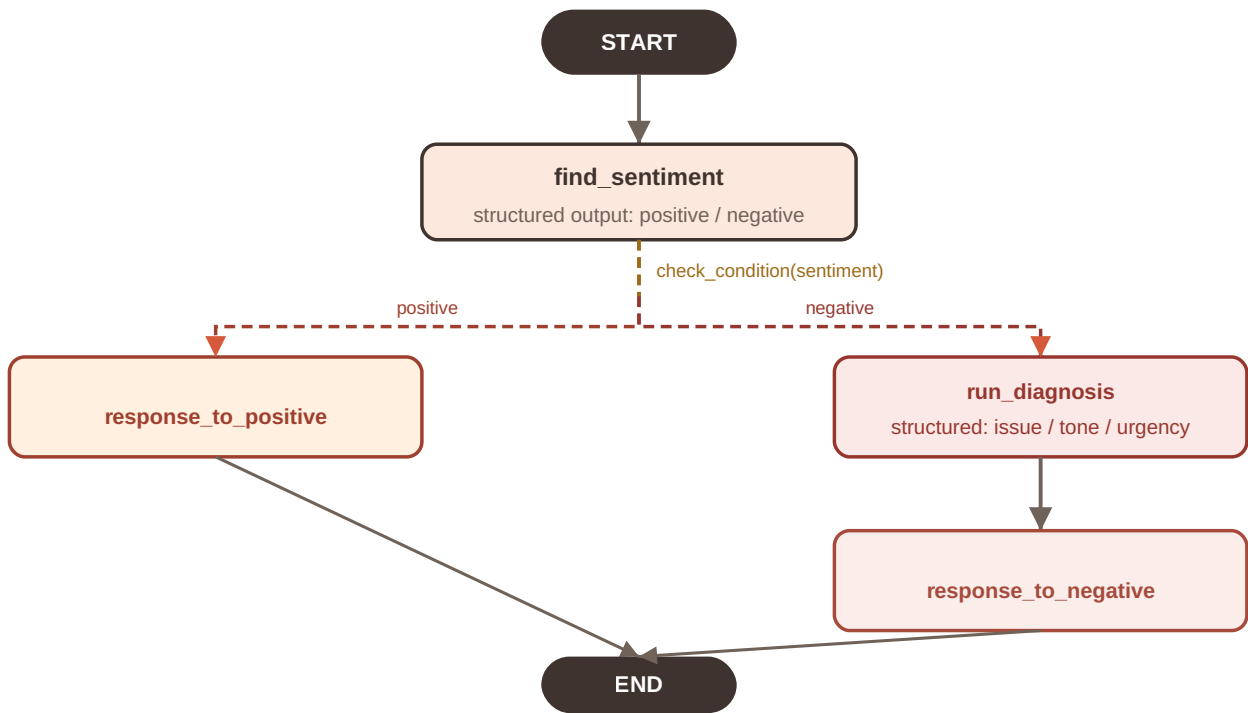


Figure 7.4 — The review-handling workflow: sentiment is classified once, then routes to a short positive-reply branch or a two-step diagnosis-then-reply branch for negative reviews.

Structured Output for Routing Decisions

Because the routing decision itself depends on an LLM's judgment, the sentiment must come back in a strictly constrained shape rather than free-form text. This is done with a Pydantic schema and `Literal`, exactly mirroring the `Literal` discipline used for routing-function return types in Section 2:

```

class Sentiment(BaseModel):
    sentiment : Literal['positive', 'negative'] = Field(description = "Sentiment of the review.")

class ReviewDiagnosis(BaseModel):
    issue_type : str = Field(description = "The issue reviewer is talking about in detail.")
    tone : str = Field(description = "The tone of the reviewer in detail.")
    urgency : str = Field(description = "The urgency of the matter for reviewer.")
  
```

Two structured models are bound to the base chat model, one per schema, so each can be invoked wherever that specific shape of output is needed:

```

model = ChatGroq(
    model="openai/gpt-oss-120b",
    temperature=0.1,    # Low temperature keeps the seller tone professional and consistent
    max_tokens=300      # Ensures the final response doesn't get truncated
)

diagnosis_structured_model = model.with_structured_output(ReviewDiagnosis)
sentiment_structured_model = model.with_structured_output(Sentiment)

```

IMPORTANT

The routing decision in this workflow is only as reliable as the structured-output schema constraining it. Using `Literal['positive', 'negative']` on the `Sentiment` schema guarantees the LLM's classification can only ever come back as one of those two exact strings — which is what makes it safe to feed directly into a routing function.

🔄 Quick Revision — Structured Output for Routing

- The routing signal here is an LLM-produced sentiment classification, not a numeric comparison.
- `Sentiment(BaseModel)` constrains output to exactly 'positive' or 'negative' via `Literal`.
- `ReviewDiagnosis(BaseModel)` extracts `issue_type`, `tone`, and `urgency` for negative reviews only.
- Two separate structured models are created from the same base `ChatGroq` model via `with_structured_output()`.
- Constraining LLM output with `Literal` is what makes it safe to use as a routing-function's decision input.

6. Building and Running the Full Review Workflow

State and Node Definitions

`ReviewState` holds the incoming review, the classified sentiment, the diagnosis details (only populated on the negative branch), and the final response text:

```

class ReviewState(TypedDict):
    review : str
    sentiment : str
    response : str
    details : Optional[dict] = None

```

The four node functions and the routing function:

```

def find_sentiment(state : ReviewState):
    prompt = f"Find the sentiment of the review : \n {state['review']}"

    response = sentiment_structured_model.invoke(prompt)

    return {"sentiment":response.sentiment}

def run_diagnosis(state : ReviewState):
    prompt = f"Fetch the issue, tone and urgency from the given review for diagnosis : \n {state['review']}."

    details = diagnosis_structured_model.invoke(prompt)

    return {"details":dict(details)}

def check_condition(state : ReviewState) -> Literal["run_diagnosis","response_to_positive"]:

    if state["sentiment"] == 'positive':
        return "response_to_positive"
    else:
        return "run_diagnosis"

```

Two Reply-Generation Nodes, Side by Side

The positive-branch reply needs no extra context beyond the review itself; the negative-branch reply is written only after the diagnosis details are available, and folds them into the prompt:

RESPONSE_TO_POSITIVE

```

def response_to_positive(state : ReviewState):
    prompt = f"""You are the seller of the
product. Generate an
appropriate response to the given positive
review.
CRITICAL INSTRUCTION: Output ONLY the final
customer
response. Do NOT include any thoughts,
reasoning,
<think> tags, or explanations. Start directly
with
the greeting.
Review:
{state['review']}"""

    response = model.invoke(prompt)

    return {"response":response.content}

```

RESPONSE_TO_NEGATIVE

```

def response_to_negative(state : ReviewState):
    prompt = f"""You are the seller of the
product. Generate an
appropriate response for the negative review.
CRITICAL INSTRUCTION: Output ONLY the final
customer
response. Do NOT include any thoughts,
reasoning,
<think> tags, or explanations. Start directly
with
the greeting.
Review:
{state['review']}
Diagnosis Details:
{state['details']}"""

    response = model.invoke(prompt)

    return {"response":response.content}

```

Assembling the Graph

```
graph = StateGraph(ReviewState)

graph.add_node("find_sentiment", find_sentiment)
graph.add_node("run_diagnosis", run_diagnosis)
graph.add_node("response_to_negative", response_to_negative)
graph.add_node("response_to_positive", response_to_positive)

graph.add_edge(START, "find_sentiment")
graph.add_conditional_edges("find_sentiment", check_condition)
graph.add_edge("run_diagnosis", "response_to_negative")
graph.add_edge("response_to_negative", END)
graph.add_edge("response_to_positive", END)

workflow = graph.compile()
```

Running the Workflow

On a positive review such as one praising a clean, intuitive interface, `find_sentiment` classifies it as **positive**, `check_condition` routes straight to `response_to_positive`, and a short thank-you message is produced. On a negative review describing a login screen that keeps freezing, the sentiment comes back **negative**, execution routes through `run_diagnosis` first (extracting an issue type such as a bug, a frustrated tone, and high urgency), and only then does `response_to_negative` draft a reply that acknowledges those specific details.

INTERVIEW TIP

A strong answer to "why not just always run the diagnosis step, even for positive reviews?" is: the diagnosis call is an extra LLM invocation with its own latency and cost. Conditional routing lets the graph skip work that adds no value for the positive path, which is exactly the efficiency benefit conditional workflows provide over a workflow that always runs every possible step.

🔄 Quick Revision — Review Workflow Assembly

- `ReviewState` tracks review, sentiment, details (negative branch only), and response.
- `check_condition` routes on `state['sentiment']` to either `response_to_positive` or `run_diagnosis`.
- `run_diagnosis` feeds into `response_to_negative` via an ordinary `add_edge`, not a second conditional edge.
- Both terminal reply nodes connect independently to `END`.
- Conditional routing avoids running the (costlier) diagnosis step on reviews that don't need it.

7. Comparing Workflow Types and What Comes Next

Comparing the Three Workflow Patterns

Aspect	Sequential	Parallel	Conditional
Branches taken per run	N/A — single path	All branches, every run	Exactly one branch, chosen at runtime
Edge type used	<code>add_edge()</code>	Multiple <code>add_edge()</code> from one fan-out node	<code>add_conditional_edges()</code> with a routing function
Decision logic	None — fixed order	None — all run	A function reading state and returning a node name
Typical use case	A fixed multi-step pipeline	Independent sub-tasks with a shared merge step	Classification, thresholds, business rules

Looking Ahead

The routing function pattern shown in this chapter is not the only way to build conditional logic in LangGraph. A second, more advanced technique uses the `Command` object to combine a state update and a routing decision inside a single node, which becomes especially useful once workflows need to be built **dynamically** rather than with a fully fixed topology. That approach is covered later in this playlist, once dynamic workflows are introduced.

🔄 Quick Revision — Workflow Pattern Comparison

- Sequential: fixed order, single path, built entirely with `add_edge()`.
- Parallel: all branches run every time, built with multiple `add_edge()` calls from one fan-out node.
- Conditional: exactly one branch runs, built with `add_conditional_edges()` plus a routing function.
- A second, more advanced way to build conditional routing exists via the `Command` object, for dynamic workflows (upcoming topic).

K. Key Takeaways

- **Conditional workflows** route execution to exactly one branch out of several, chosen at runtime — the LangGraph equivalent of `if / elif / else`, unlike parallel workflows where every branch always runs.
- **Routing functions** are plain functions that take state and return the string name of the next node; annotate their return type with `Literal[...]` to document and validate every possible branch.
- `graph.add_conditional_edges(source, routing_fn)` registers a routing function against a source node, replacing what would otherwise require several manual conditional `add_edge()` calls.

- The **quadratic equation workflow** demonstrated a non-LLM conditional workflow: the sign of the discriminant routes execution to `real_roots`, `repeated_roots`, or `no_roots`.
- Watch for **operator-precedence bugs** when translating math formulas with compound denominators (e.g. writing `x/2*a` instead of `x/(2*a)`) directly into Python.
- The **review-handling workflow** demonstrated an LLM-based conditional workflow: a structured-output sentiment classification routes execution to a short positive-reply branch or a longer diagnose-then-reply negative branch.
- **Structured output with Literal** schemas (`Sentiment`, `ReviewDiagnosis`) constrains LLM output into a shape that is safe to feed directly into routing decisions.
- Conditional workflows let a graph **skip unnecessary work** — only the relevant branch's LLM calls or computation actually run on a given invocation.
- A second, more advanced technique for conditional routing uses the **Command** object, reserved for when dynamic workflows are introduced later in the playlist.

R. Revision Sheet

Conditional Workflow

A graph that executes exactly one of several possible branches per run, chosen at runtime based on state.

Sentiment Schema

`Sentiment(BaseModel)` with `sentiment: Literal['positive','negative']` constrains LLM output for safe routing.

Routing Function

A function taking state as input and returning the string name of the next node; typed with `Literal[...]` for every valid destination.

Diagnosis Schema

`ReviewDiagnosis(BaseModel)` extracts `issue_type`, `tone`, and `urgency` — only computed on the negative branch.

add_conditional_edges()

`graph.add_conditional_edges(source_node, routing_fn)` — registers a routing function as the decision logic leaving a source node.

Conditional vs. Parallel

Conditional: exactly one branch runs. Parallel: every branch runs, every time.

Discriminant Routing

$D > 0 \rightarrow \text{real_roots}$, $D == 0 \rightarrow \text{repeated_roots}$, $D < 0 \rightarrow \text{no_roots}$; a clean non-LLM branching example.

Command Object (preview)

A more advanced alternative for building conditional routing, used for dynamic workflows in a later chapter.

I. Interview Questions

Q1. What is a conditional workflow in LangGraph, and how does it differ conceptually from a parallel workflow?

Q2. What is a routing function, and what must its return value match?

Q3. Why is it good practice to annotate a routing function's return type with `Literal[...]`?

Q4. What two arguments does `graph.add_conditional_edges()` take, and what does each control?

Q5. In the quadratic-equation workflow, which node computes the value used to decide the branch, and which node consumes that decision?

Q6. Explain the operator-precedence bug that can occur when translating the quadratic formula's $/2a$ denominator into Python.

Q7. How is structured output used to make an LLM's sentiment classification safe to route on?

Q8. Why does the review-handling workflow run a diagnosis step only for negative reviews and not for positive ones?

Q9. Can a conditional workflow have more than two branches? Justify your answer with reference to the quadratic-equation example.

Q10. What alternative to routing functions exists for building conditional logic in LangGraph, and when does the lecture indicate it becomes especially useful?

F. Further Reading

- LangGraph official documentation — "Add conditional edges" (graph construction guide).
- LangGraph official documentation — Command object reference, for dynamic routing and state updates.
- Python `typing.Literal` documentation — for precise return-type contracts on routing functions.
- Pydantic documentation — `BaseModel` and `Field`, for defining structured-output schemas.
- NCERT Class 10 Mathematics — Quadratic Equations chapter, for a refresher on the discriminant and root formulas.